

# Task 1: Weak vs Strong Prompt Design

| Weak Prompt                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Strong Prompt                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>Tell me about cloud computing</div><p>Cloud computing is a technology that allows you to access and use computing resources—like servers, storage, databases, networking, and software—over the internet instead of relying on your personal computer or local servers.</p><p><b>Simple Idea</b></p><p>Instead of storing files or running programs on your own device, you use remote servers hosted by companies like Amazon Web Services, Microsoft Azure, or Google Cloud Platform.</p><p><b>Key Features</b></p><ul style="list-style-type: none"><li>• <b>On-demand access</b> – Use resources whenever you need</li><li>• <b>Scalability</b> – Increase or decrease resources easily</li><li>• <b>Cost-effective</b> – Pay only for what you use</li><li>• <b>Remote access</b> – Access from anywhere with internet</li></ul></div> | <div><div>Explain cloud computing to a 2nd-year engineering student. Include:<br/><br/>A clear definition<br/><br/>Types of cloud services (IaaS, PaaS, SaaS) with examples<br/><br/>Deployment models (public, private, hybrid)<br/><br/>Real-world use cases<br/><br/>Advantages and disadvantages<br/>Format the answer in structured bullet points and keep it under 300 words.</div><p><b>Cloud Computing (For a 2nd-Year Engineering Student)</b></p><p><b>Definition</b></p><p>Cloud computing is the delivery of computing services (servers, storage, databases, networking, software) over the internet, allowing users to access resources on-demand without owning physical hardware.</p><p>↓</p></div> <div><p><b>Types of Cloud Services</b></p><ul style="list-style-type: none"><li>• <b>IaaS (Infrastructure as a Service)</b><ul style="list-style-type: none"><li>• Provides virtualized hardware (VMs, storage, networks)</li><li>• Full control over OS and applications</li><li>• Example: Amazon Web Services EC2</li></ul></li><li>• <b>PaaS (Platform as a Service)</b><ul style="list-style-type: none"><li>• Provides a platform to develop, run, and deploy applications</li><li>• No need to manage underlying hardware</li><li>• Example: Google Cloud Platform App Engine</li></ul></li><li>• <b>SaaS (Software as a Service)</b><ul style="list-style-type: none"><li>• Ready-to-use software via the internet</li><li>• No installation required</li><li>• Examples: Google Docs, Microsoft 365</li></ul></li></ul></div> <div><p><b>Deployment Models</b></p><ul style="list-style-type: none"><li>• <b>Public Cloud</b><ul style="list-style-type: none"><li>• Services offered over the internet to multiple users</li><li>• Example: Microsoft Azure</li></ul></li><li>• <b>Private Cloud</b><ul style="list-style-type: none"><li>• Dedicated to a single organization</li><li>• More control and security</li></ul></li><li>• <b>Hybrid Cloud</b><ul style="list-style-type: none"><li>• Combination of public and private cloud</li><li>• Balances flexibility and security</li></ul></li></ul><p><b>Real-World Use Cases</b></p><ul style="list-style-type: none"><li>• Hosting websites and web apps</li><li>• Data storage and backup (e.g., Google Drive)</li><li>• Running AI/ML models</li><li>• Streaming services (Netflix, Spotify)</li><li>• Online collaboration tools</li></ul><p>↓</p></div> |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div>♦ <b>Advantages</b></div> <ul style="list-style-type: none"><li>• Scalable and flexible</li><li>• Cost-efficient (pay-as-you-go)</li><li>• Remote accessibility</li><li>• High availability and backup</li></ul> <div>♦ <b>Disadvantages</b></div> <ul style="list-style-type: none"><li>• Internet dependency</li><li>• Security and privacy concerns</li><li>• Downtime risks</li><li>• Limited control in some models</li></ul> |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Comparison :

Weak Output:

- Generic definition
- No structure
- Missing examples

Strong Output:

- Well-organized
- Includes technical depth + examples
- Easy to revise before exams

| Weak Prompt                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Strong Prompt                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>Write a message</div><div>Got it—but I need a bit more context 🤖<br/>What kind of message do you want?</div><div>For now, here are a few common types you might be looking for:</div><div>1. Formal message (professional):<br/>"Dear Sir/Ma'am,<br/>I hope this message finds you well. I am writing to inform you regarding [purpose]. Kindly let me know a convenient time to discuss this further.<br/>Thank you for your time and consideration.<br/>Regards."</div><div>2. Friendly/casual message:<br/>"Hey! Just wanted to check in and see how you're doing 🍌 Let me know when you're free—we should catch up soon!"</div><div>3. Appreciation message:<br/>"I just wanted to take a moment to thank you for your 👇 support and guidance. It truly means a lot to me, and I'm really grateful."</div></div> | <div><div>Write a formal email to a recruiter asking if technical roles are available instead of a founder-led sales role.Keep the tone polite and professional.<br/>Include:<br/>A brief introduction<br/>Interest in the company<br/>Clear question about technical opportunities<br/>Proper closing<br/>Limit the email to 120-150 words.</div><div>Email<br/>Subject Inquiry Regarding Technical Role Opportunities<br/><br/>Dear [Recruiter's Name],<br/><br/>I hope this message finds you well. My name is Anushka Ekka, and I recently came across the opportunity for a founder-led sales role at your organization.<br/><br/>I would like to express my genuine interest in your 👇 mpany and the impactful work it is doing. However, given my background and academic focus in Data Science and Artificial Intelligence, I am</div></div> |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | <div>Email<br/>Subject Inquiry Regarding Technical Role Opportunities<br/><br/>Dear [Recruiter's Name],<br/><br/>I hope this message finds you well. My name is Anushka Ekka, and I recently came across the opportunity for a founder-led sales role at your organization.<br/><br/>I would like to express my genuine interest in your company and the impactful work it is doing. However, given my background and academic focus in Data Science and Artificial Intelligence, I am particularly inclined toward technical roles.<br/><br/>I wanted to kindly inquire if there are any current or upcoming opportunities in technical domains where my skills and interests may be better aligned.<br/><br/>Thank you for your time and consideration. I look forward to your response.<br/><br/>Warm regards,<br/>Anushka Ekka</div>             |

Comparison:

- **Weak Output:**
  - Ambiguous (could be anything: SMS, casual note, etc.)
- **Strong Output:**
  - Clear format (email)
  - Correct tone (professional)
  - Directly usable

Weak Prompt

Explain binary search

🔍 Binary Search (Simple Explanation)

Binary Search is an efficient algorithm used to find an element in a **sorted array** by repeatedly dividing the search space in half.

💡 Key Idea

Instead of checking every element (like linear search), binary search:

- Compares the target with the **middle element**
- Decides whether to search in the **left half** or **right half**

Strong Prompt

Explain the binary search algorithm to a beginner in computer science. Include:  
Step-by-step explanation  
A simple example with numbers  
Time and space complexity  
C++ code implementation  
Format the explanation clearly with headings and code blocks.

🔍 Binary Search Algorithm (Beginner-Friendly)

Binary Search is a fast way to find an element in a **sorted array** by repeatedly cutting the search space in half.

⚙️ Step-by-Step Explanation

1. Start with a sorted array

Binary search only works if the data is sorted.

2. Initialize two pointers:

- `low = 0` (start of array)
- `high = n - 1` (end of array)

3. Find the middle element: ↓

3. Find the middle element:

`mid = (low + high) / 2`

📄

4. Compare the middle element with the target:

- If `arr[mid] == target` → ☒ Element found
- If `target < arr[mid]` → Search **left half**  
→ update `high = mid - 1`
- If `target > arr[mid]` → Search **right half**  
→ update `low = mid + 1`

5. Repeat steps 3–4 until:

- Element is found OR
- `low > high` (element not present)

🎯 Simple Example

Array: `[1, 3, 5, 7, 9, 11]`

Target: `7`

| Step | low | high | mid | arr[mid] | Action                                    |
|------|-----|------|-----|----------|-------------------------------------------|
| 1    | 0   | 5    | 2   | 5        | Go right                                  |
| 2    | 3   | 5    | 4   | 9        | Go left                                   |
| 3    | 3   | 3    | 3   | 7        | <input checked="" type="checkbox"/> Found |

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <div><div>C++ Code Implementation</div><pre>1 // C++ 2 #include &lt;iostream&gt; 3 using namespace std; 4 5 int binarySearch(int arr[], int n, int target) { 6     int low = 0, high = n - 1; 7 8     while (low &lt;= high) { 9         int mid = (low + high) / 2; 10 11         if (arr[mid] == target) 12             return mid; 13         else if (target &lt; arr[mid]) 14             high = mid - 1; 15         else 16             low = mid + 1; 17     } 18 19     return -1; // Element not found 20 } 21 22 int main() { 23     int arr[] = {1, 3, 5, 7, 9, 11}; 24     int n = sizeof(arr) / sizeof(arr[0]); 25     int target = 7; 26 27     int result = binarySearch(arr, n, target); 28 29     if (result != -1) 30         cout &lt;&lt; "Element found at index: " &lt;&lt; result; 31     else 32         cout &lt;&lt; "Element not found"; 33 34     return 0; 35 }</pre></div> |
|  | <div><div><div>Time Complexity</div><ul style="list-style-type: none"><li>• <b>Best Case:</b> <math>O(1)</math> → element found immediately</li><li>• <b>Worst Case:</b> <math>O(\log n)</math> → search space halves each step</li></ul><p>👉 Very efficient for large datasets.</p></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|  | <div><div><div>Space Complexity</div><ul style="list-style-type: none"><li>• <b>Iterative version:</b> <math>O(1)</math> (no extra space used)</li><li>• <b>Recursive version:</b> <math>O(\log n)</math> (due to function call stack)</li></ul><div><div>Important Notes</div><ul style="list-style-type: none"><li>• The array must be sorted</li><li>• Works only on random access structures (like arrays, not linked lists)</li></ul></div></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                             |

Comparison:

- **Weak Output:**
  - Only theory
  - No code or example
- **Strong Output:**
  - Full concept + implementation
  - Better for interviews and coding
  - Even tells the constraints of applying

|             |               |
|-------------|---------------|
| Weak Prompt | Strong Prompt |
|-------------|---------------|

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>Tell me about machine learning</div><div>Here's a clear and structured explanation of <b>Machine Learning (ML)</b> tailored for you 🗨️</div><div><div>♦ <b>What is Machine Learning?</b></div><div>Machine Learning is a branch of Artificial Intelligence that enables computers to <b>learn patterns from data and make decisions or predictions without being explicitly programmed.</b></div><div>🗨️ In simple words:<br/>Instead of writing rules manually, we give data → the system learns → and then makes predictions.</div></div></div> | <div><div><div>Provide an overview of machine learning for a college student.<br/>Include:<br/>Definition and how it works<br/>Types<br/>Common algorithms<br/>Real-world applications<br/>Pros and limitations<br/>Keep the explanation simple but informative, and structure it with headings and bullet points.</div><div>Here's a simple, structured overview of <b>Machine Learning (ML)</b> for a college student 🗨️</div><div><div>♦ <b>Definition of Machine Learning</b></div><div>Machine Learning is a subfield of Artificial Intelligence where computers <b>learn from data to make predictions or decisions without being explicitly programmed.</b></div><div>🗨️ Instead of writing rules, we train models using data.</div></div></div></div>                                                                                                                                    |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <div><div><div>♦ <b>How Machine Learning Works</b></div><div><ul style="list-style-type: none"><li>📁 <b>Data Collection</b> – Gather relevant data</li><li>🔧 <b>Data Preprocessing</b> – Clean and organize data</li><li>📊 <b>Model Selection</b> – Choose an algorithm</li><li>🔗 <b>Training</b> – Model learns patterns from data</li><li>📈 <b>Testing/Evaluation</b> – Check performance on new data</li><li>🎯 <b>Prediction</b> – Use model for real-world tasks</li></ul></div><div><div>♦ <b>Types of Machine Learning</b></div><div><div>1. <b>Supervised Learning</b></div><div><ul style="list-style-type: none"><li>• Uses <b>labeled data</b></li><li>• Predicts outputs from inputs</li><li>• <b>Examples:</b> Spam detection, price prediction</li></ul></div></div></div></div></div>                                                                                              |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <div><div><div>2. <b>Unsupervised Learning</b></div><div><ul style="list-style-type: none"><li>• Uses <b>unlabeled data</b></li><li>• Finds hidden patterns or groups</li><li>• <b>Examples:</b> Customer segmentation</li></ul></div><div><div>3. <b>Reinforcement Learning</b></div><div><ul style="list-style-type: none"><li>• Learns through <b>rewards and penalties</b></li><li>• <b>Examples:</b> Game playing, robotics</li></ul></div><div><div>♦ <b>Common Algorithms</b></div><div><ul style="list-style-type: none"><li>📈 <b>Linear Regression</b> – Predicts continuous values</li><li>🌳 <b>Decision Trees</b> – Tree-based decision making</li><li>📁 <b>K-Means Clustering</b> – Groups similar data</li><li>🧠 <b>Neural Networks</b> – Inspired by the human brain</li><li>🔍 <b>Support Vector Machines (SVM)</b> – Classification tasks</li></ul></div></div></div></div></div> |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <div><div><div>♦ <b>Real-World Applications</b></div><div><ul style="list-style-type: none"><li>📺 <b>Recommendations</b> (Netflix, YouTube)</li><li>🛒 <b>E-commerce suggestions</b></li><li>🏥 <b>Disease prediction</b> in healthcare</li><li>🚗 <b>Self-driving cars</b></li><li>🗣️ <b>Voice assistants</b> like Google Assistant and Siri</li></ul></div><div><div>♦ <b>Advantages (Pros)</b></div><div><ul style="list-style-type: none"><li>✅ Automates complex tasks</li><li>✅ Improves accuracy over time</li><li>✅ Can handle large datasets</li></ul></div><div><div>♦ <b>Limitations (Cons)</b></div><div><ul style="list-style-type: none"><li>❌ Requires large amounts of data</li><li>❌ Can be biased if data is biased</li><li>❌ High computational cost</li><li>❌ Sometimes hard to interpret (black-box mod. ⬇️)</li></ul></div></div></div></div></div>                           |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | <div><div><div>♦ <b>Final Thought</b></div><div>Machine Learning is a powerful tool that is transforming industries, but it requires <b>good data, proper models, and careful evaluation</b> to work effectively.</div></div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

Comparison:

- **Weak Output:**
  - Surface-level explanation
- **Strong Output:**
  - Covers concepts + applications
  - More structured and exam-ready

## Final Insight

A strong prompt usually includes:

1. Audience specification
2. Context definition
3. Specific content requirements
4. Output format
5. Constraints and guidelines

## Task 2: Role-Based Prompting

### 1. Teacher Role Prompt

Act as a teacher and explain the concept of Binary Search to a beginner student. Include a simple step-by-step explanation, a basic example, and a clear conclusion. Keep the language easy to understand.

Act as a teacher and explain the concept of Binary Search to a beginner student. Include a simple step-by-step explanation, a basic example, and a clear conclusion. Keep the language easy to understand.

Alright, let's learn **Binary Search** in a simple and clear way 📖

---

🌟 **What is Binary Search?**

Binary Search is a **fast way to find an element in a sorted list (array)**.

🔥 Important: The list **must be sorted** (either ascending or descending).

Instead of checking every element one by one, Binary Search:

- Repeatedly **divides the list into halves**
- Checks the **middle element**
- Decides which half to continue searching in

**Step-by-Step Explanation**

Let's say we want to find a number in a sorted array.

**Steps:**

1. Find the **middle element** of the array
2. Compare it with the target value
  - If it is equal → 🟢 Found
  - If target is smaller → search in the **left half**
  - If target is larger → search in the **right half**
3. Repeat the process on the remaining half
4. Stop when the element is found or the search space becomes empty

**Simple Example**

Let's search for 7 in this array:

[1, 3, 5, 7, 9, 11]

Step 1:  
Middle element = 5  
7 > 5 → Search in right half

[7, 9, 11]

Step 2:  
Middle element = 9  
7 < 9 → Search in left half

[7]

Step 3:  
Middle element = 7  
Found! 🟢

### 🕒 Why is Binary Search Fast?

- Each step **cuts the array into half**
- So fewer comparisons are needed

👉 Time Complexity:  $O(\log n)$

👉 Much faster than linear search ( $O(n)$ )

### 🎯 Conclusion

- Binary Search is an **efficient searching algorithm**
- It works only on **sorted data**
- It reduces the search space by **half each time**
- This makes it very useful for **large datasets**

## 2. Software Engineer Role Prompt

Act as a software engineer and explain Binary Search. Include the algorithm logic, time and space complexity, edge cases, and provide an efficient C++ implementation. Keep the explanation concise and technical.

Act as a software engineer and explain Binary Search. Include the algorithm logic, time and space complexity, edge cases, and provide an efficient C++ implementation. Keep the explanation concise and technical.

### 🔍 Binary Search (Software Engineer Perspective)

#### ✅ Core Idea

Binary Search operates on a **sorted array** and repeatedly halves the search space to locate a target value.

#### ⚙️ Algorithm Logic

1. Initialize two pointers:
  - `low = 0`
  - `high = n - 1`
2. While `low <= high`:
  - Compute middle index:

2. While `low <= high`:

- Compute middle index:

```
C++
mid = low + (high - low) / 2;
```

- Compare `arr[mid]` with target:

- If equal → return `mid`
- If target < `arr[mid]` → search left half → `high = mid - 1`
- Else → search right half → `low = mid + 1`

3. If loop ends → target not found → return `-1`

#### 🔧 Complexity Analysis

- Time Complexity:
  - Best case:  $O(1)$  (middle element matches)
  - Average/Worst case:  $O(\log n)$
- Space Complexity:
  - Iterative:  $O(1)$
  - Recursive:  $O(\log n)$  (due to call stack)

#### ⚠ Edge Cases

- Empty array → return -1
- Single element array
- Target not present
- Duplicate elements → may return any valid index
- Integer overflow when computing `mid`
  - ✓ Use: `low + (high - low) / 2`
- Unsorted array → ✗ Binary Search not applicable

#### 📦 Efficient C++ Implementation (Iterative)

```
C++
#include <bits/stdc++.h>
using namespace std;

int binarySearch(vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
```

```
C++

        if (arr[mid] == target)
            return mid;
        else if (arr[mid] > target)
            high = mid - 1;
        else
            low = mid + 1;
    }

    return -1; // target not found
}

int main() {
    vector<int> arr = {1, 3, 5, 7, 9, 11};
    int target = 7;

    int result = binarySearch(arr, target);

    if (result != -1)
        cout << "Element found at index: " << result << endl;
    else
        cout << "Element not found" << endl;

    return 0;
}
```

## 🧠 Summary

- Requires sorted data
- Reduces search space by half each iteration
- Highly efficient for large datasets
- Prefer iterative version for constant space usage

### 3. Researcher Role Prompt

Act as a computer science researcher and explain Binary Search. Discuss its theoretical foundation, algorithmic efficiency, comparisons with other search algorithms, and scenarios where it performs optimally. Use a formal and analytical tone.



Act as a computer science researcher and explain Binary Search. Discuss its theoretical foundation, algorithmic efficiency, comparisons with other search algorithms, and scenarios where it performs optimally. Use a formal and analytical tone.

Binary Search: A Research-Oriented Explanation

1. Theoretical Foundation

Binary Search is grounded in the **divide-and-conquer paradigm**, where a problem is recursively reduced into smaller subproblems. Given a **totally ordered set** (i.e., a sorted array), Binary Search exploits order to eliminate half of the search space at each step.

Formally, for a sorted array  $A$  of size  $n$ , the algorithm compares the target value  $x$  with the middle element  $A[\lfloor n/2 \rfloor]$ , and based on the comparison, restricts the search to either the left or right subarray. This halving process continues until the element is found or the subarray becomes empty.

This behavior corresponds to a **binary decision tree**, where each comparison reduces the uncertainty by one bit. Hence, Binary Search is optimal among comparison-based search algorithms on sorted data.

|         |              |           |                                            |
|---------|--------------|-----------|--------------------------------------------|
| Hashing | $O(1)$ (avg) | Unordered | Requires extra space, not comparison-based |
|---------|--------------|-----------|--------------------------------------------|

Binary Search is superior to Linear and Jump Search in asymptotic efficiency, but unlike Hashing, it does not require additional memory. However, it is constrained by the prerequisite of sorted data and efficient random access.

4. Optimal Scenarios for Binary Search

Binary Search performs optimally under the following conditions:

- **Static, sorted datasets:** When the data does not change frequently, allowing the cost of sorting to be amortized.
- **Random access structures:** Such as arrays, where index-based access is constant time.
- **Large datasets:** Where  $O(\log n)$  performance provides significant gains over linear methods.
- **Low memory environments:** Where in-place searching (no extra space) is preferred over hashing.
- **Monotonic search spaces:** It is widely used beyond arrays, e.g., in **binary search on answer**, where the solution space is monotonic.

2. Algorithmic Efficiency

- **Time Complexity:**
  - Worst-case:  $O(\log n)$
  - Average-case:  $O(\log n)$
  - Best-case:  $O(1)$
- **Space Complexity:**
  - Iterative implementation:  $O(1)$
  - Recursive implementation:  $O(\log n)$  due to stack depth

From an information-theoretic perspective, Binary Search achieves the lower bound of  $\Omega(\log n)$  comparisons required to locate an element in a sorted list, making it asymptotically optimal.

3. Comparison with Other Search Algorithms

| Algorithm            | Time Complexity        | Data Requirement        | Remarks                       |
|----------------------|------------------------|-------------------------|-------------------------------|
| Linear Search        | $O(n)$                 | Unsorted/Sorted         | No preprocessing required     |
| Binary Search        | $O(\log n)$            | Sorted                  | Requires random access        |
| Jump Search          | $O(\sqrt{n})$          | Sorted                  | Fewer comparisons than linear |
| Interpolation Search | $O(\log \log n)$ (avg) | Sorted (uniformly)<br>↓ | Degrades to $O(n)$            |

5. Limitations

- Ineffective on **unsorted data**
- Not suitable for **linked lists** (lack of random access)
- Performance depends on **data layout and cache locality**
- Requires careful handling of **edge conditions** (e.g., overflow, duplicates)

6. Conclusion

Binary Search is a fundamental algorithm that exemplifies the power of exploiting structure (ordering) to achieve optimal efficiency. Its  $O(\log n)$  time complexity aligns with the theoretical lower bound for comparison-based search, making it a cornerstone in algorithm design. Despite its constraints, it remains indispensable in both theoretical computer science and practical system implementations, particularly in scenarios involving large, static, and ordered datasets.

Observations

1. Explanation Style

- Teacher: Simple, step-by-step, easy to understand
- Software Engineer: Practical, implementation-focused
- Researcher: Formal, analytical, theory-oriented

2. Technical Depth

- Teacher: Low to moderate
- Software Engineer: High (includes code and edge cases)
- Researcher: Very high (focus on theory and comparisons)

3. Suitability for Audience

- Teacher: Best for beginners and students
- Software Engineer: Best for coding practice and interviews
- Researcher: Best for academic and in-depth understanding

## Task 3: Adding Constraints to Prompts

### Prompts

#### 1. Artificial Intelligence (Bullet Points with Length Constraint)

Explain Artificial Intelligence in exactly 5 bullet points. Keep each point concise (1–2 lines) and include definition, types, applications, advantages, and limitations.

Explain Artificial Intelligence in exactly 5 bullet points. Keep each point concise (1–2 lines) and include definition, types, applications, advantages, and limitations.

- **Definition:** Artificial Intelligence (AI) is the simulation of human intelligence in machines, enabling them to learn, reason, and make decisions.
- **Types:** AI is categorized into Narrow AI (task-specific), General AI (human-level intelligence), and Superintelligent AI (beyond human capability).
- **Applications:** Used in healthcare (diagnosis), finance (fraud detection), education (personalized learning), and automation (self-driving cars, chatbots).
- **Advantages:** Improves efficiency, handles repetitive tasks, reduces human error, and enables data-driven decision-making.
- **Limitations:** Requires large data, can be biased, lacks true human understanding, and raises ethical and job displacement concerns.

#### 2. Binary Search (Step-by-Step with Example Constraint)

Explain the Binary Search Algorithm with a clear step-by-step explanation. Include a numerical example, show each iteration, and mention time complexity. Present the answer in numbered steps.

👉 Element found at index 3

 (For better understanding, let's assume target was not found immediately)

If target was 10, steps would continue like this:

### 3. Iteration 1 (target = 10)

- low = 0, high = 6
- mid = 3, element = 8
- 10 > 8 → Search right half
- Update: low = 4

#### 4. Iteration 2

- low = 4, high = 6
- mid = (4 + 6) / 2 = 5, element = 12
- 10 < 12 → Search left half
- Update: high = 4

- Compute mid: `mid = (0 + 6) / 2 = 3`
- Element at mid: `8`
- Compare: `8 == 8` 

- `low = 4, high = 4`
- `mid = 4, element = 10`
- `10 == 10` 

### Time Complexity

- Best case:  $O(1)$  (element found at middle)
- Worst/Average case:  $O(\log n)$  (search space halves each step)

7. Binary Search works only on **sorted arrays** and efficiently reduces the search space by half in each iteration, making it much faster than linear search for large datasets.

Explain common Sorting Algorithms in a table format. Include columns for Algorithm Name, Time Complexity (Best/Average/Worst), Space Complexity, and Key Features. Cover at least 4 algorithms.



| Algorithm Name | Time Complexity (Best / Avg / Worst)      | Space Complexity                                                                                   | Key Features                                                             |
|----------------|-------------------------------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Bubble Sort    | $O(n) / O(n^2) / O(n^2)$                  | $O(1)$                                                                                             | Simple, stable, in-place; inefficient for large datasets                 |
| Selection Sort | $O(n^2) / O(n^2) / O(n^2)$                | $O(1)$                                                                                             | In-place, not stable; minimal swaps but always quadratic time            |
| Insertion Sort | $O(n) / O(n^2) / O(n^2)$                  | $O(1)$                                                                                             | Efficient for small or nearly sorted data; stable and adaptive           |
| Merge Sort     | $O(n \log n) / O(n \log n) / O(n \log n)$ | $O(n)$                                                                                             | Stable, divide-and-conquer; good for large datasets but uses extra space |
| Quick Sort     | $O(n \log n) / O(n \log n) / O(n^2)$      | $O(\log n)$<br> | Fast in practice, in-place; not stable, worst-case depends on pivot      |
| Heap Sort      | $O(n \log n) / O(n \log n) / O(n \log n)$ | $O(1)$                                                                                             | In-place, not stable; based on heap data structure                       |

# Observations

## 1. Clarity

- Constraints improve clarity by forcing concise and focused responses.
- Bullet points and steps make information easier to understand.

## 2. Organization

- Format constraints (bullets, steps, tables) lead to well-structured outputs.
- Information becomes more readable and logically arranged.

## 3. Completeness

- Content constraints ensure all important aspects are covered.
- However, strict limits (like fixed bullet points) may slightly reduce depth.

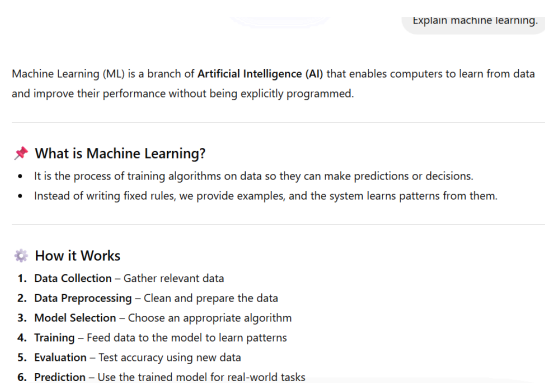
## 4. Consistency

- Responses follow a predictable structure.
- Easier to compare and review outputs.

# Task 4: Iterative Prompting

## Step 1: Original Prompt

Explain machine learning.



The screenshot shows a chat interface. At the top, a light blue bubble contains the prompt "explain machine learning.". Below it, a light green bubble contains the response. The response starts with a definition of Machine Learning (ML) as a branch of Artificial Intelligence (AI). It then has two sections: "What is Machine Learning?" with two bullet points, and "How it Works" with a numbered list of six steps: Data Collection, Data Preprocessing, Model Selection, Training, Evaluation, and Prediction.

explain machine learning.

Machine Learning (ML) is a branch of **Artificial Intelligence (AI)** that enables computers to learn from data and improve their performance without being explicitly programmed.

★ **What is Machine Learning?**

- It is the process of training algorithms on data so they can make predictions or decisions.
- Instead of writing fixed rules, we provide examples, and the system learns patterns from them.

⚙️ **How it Works**

1. **Data Collection** – Gather relevant data
2. **Data Preprocessing** – Clean and prepare the data
3. **Model Selection** – Choose an appropriate algorithm
4. **Training** – Feed data to the model to learn patterns
5. **Evaluation** – Test accuracy using new data
6. **Prediction** – Use the trained model for real-world tasks

## Output (Summary)

- Basic definition of machine learning

- General explanation
- Lacks examples and structure
- Limited depth

## Step 2: Improved Prompt (Adding Context)

Explain machine learning to a college student. Include definition, types (supervised, unsupervised, reinforcement), and simple examples.

Explain machine learning to a college student. Include definition, types (supervised, unsupervised, reinforcement), and simple examples.

Here's a clear and student-friendly explanation of Machine Learning:

**Definition**

**Machine Learning (ML)** is a branch of Artificial Intelligence where computers learn patterns from data and use them to make predictions or decisions—without being explicitly programmed for every task.

In simple terms:  
Instead of writing rules, you give data, and the system "learns" the rules.

**Types of Machine Learning**

**1. Supervised Learning**

- The model learns from **labeled data** (input + correct output already known).
- Goal: Learn a mapping from input → output.

**2. Unsupervised Learning**

- The model works with **unlabeled data** (no correct answers given).
- Goal: Find hidden patterns or groupings.

**3. Reinforcement Learning**

- The model learns by **trial and error** using rewards and penalties.
- Goal: Maximize total reward over time.

**Example:**

- Training a robot to walk
- AI playing games like chess or video games

Like learning through experience (rewards = good, penalties = bad).

**Quick Summary Table**

| Type                   | Data Type    | Goal                 | Example                |
|------------------------|--------------|----------------------|------------------------|
| Supervised Learning    | Labeled      | Predict output       | House price prediction |
| Unsupervised Learning  | Unlabeled    | Find patterns/groups | Customer segmentation  |
| Reinforcement Learning | Reward-based | Learn via feedback   | Game-playing AI        |

**Example:**

- Predicting house prices
  - Input: size, location
  - Output: price
- Email spam detection (spam or not spam)

Like studying with answer keys.

**2. Unsupervised Learning**

- The model works with **unlabeled data** (no correct answers given).
- Goal: Find hidden patterns or groupings.

**Example:**

- Customer segmentation in marketing
- Grouping similar news articles

Like finding patterns without being told the answers.

**Final Intuition**

- Supervised** → Learn with answers
- Unsupervised** → Learn without answers
- Reinforcement** → Learn from feedback

## Output (Summary)

- Clear definition
- Includes types of machine learning
- Provides basic examples
- Better structured and more informative

## Step 3: Further Improved Prompt (Adding Specificity and Constraints)

Explain machine learning to a 2nd-year engineering student. Include:

- Definition
- Types (supervised, unsupervised, reinforcement learning)
- Real-world examples from healthcare and finance
- Advantages and limitations

Present the answer in bullet points and keep it within 200–250 words.

Explain machine learning to a 2nd-year engineering student. Include:  
Definition  
Types (supervised, unsupervised, reinforcement learning)  
Real-world examples from healthcare and finance  
Advantages and limitations  
Present the answer in bullet points and keep it within 200–250 words

#### ✦ Definition

- **Machine Learning (ML)** is a subset of Artificial Intelligence where systems learn patterns from data and make predictions or decisions without being explicitly programmed.
- It focuses on building models that improve automatically with experience.

#### 🔍 Types of Machine Learning

- **Supervised Learning**
  - Trained on labeled data (input + correct output).
  - Used for prediction and classification tasks.
  - *Example:* Predicting disease risk based on patient data.

#### • Unsupervised Learning

- Works on unlabeled data to find hidden patterns or groups.
- Useful for clustering and data exploration.
- *Example:* Grouping customers based on spending behavior.

#### • Reinforcement Learning

- Learns through trial and error using rewards and penalties.
- Focuses on decision-making over time.
- *Example:* Training AI agents for automated trading or robotics.

#### 🌐 Real-World Examples

##### • Healthcare

- Disease prediction (e.g., diabetes detection)
- Medical image analysis (tumor detection in scans)

##### • Finance

- Fraud detection in transactions
- Stock price prediction and algorithmic trading

#### ✅ Advantages

- Automates complex decision-making
- Improves accuracy with more data
- Handles large and high-dimensional datasets

#### ⚠️ Limitations

- Requires large, high-quality data
- Can be biased if training data is biased
- Computationally expensive
- Some models lack interpretability (black-box nature)

## Final Output (Summary)

- Well-structured in bullet points
- Includes domain-specific examples
- Covers advantages and limitations
- Concise and easy to understand
- Highly relevant for academic use

## Observations

### 1. Improved Clarity

- Each refinement makes the response clearer and more focused.

## 2. Better Structure

- Adding format constraints results in organized outputs.

## 3. Increased Relevance

- Specifying audiences and domains makes the content more useful.

## 4. Greater Completeness

- Additional instructions ensure all important aspects are covered.

## 5. Controlled Length and Quality

- Word limits prevent unnecessary information and improve readability.

# Conclusion

Iterative prompting significantly enhances the quality of AI-generated responses. By gradually adding context, structure, and constraints, the output becomes more precise, organized, and suitable for the intended purpose.

## Task 5: Prompt for Code Generation

### 1. Prompt for Binary Search

Write a C++ program to implement Binary Search. Include detailed comments explaining each step of the algorithm. Also, mention the time and space complexity.

Write a C++ program to implement Binary Search. Include detailed comments explaining each step of the algorithm. Also, mention the time and space complexity.

Here's a well-structured C++ program for Binary Search with detailed comments explaining each step:

Binary Search in C++

```

C++
#include <iostream>
using namespace std;

// Function to perform Binary Search
int binarySearch(int arr[], int size, int target) {
    int left = 0;           // Starting index
    int right = size - 1;    // Ending index

    // Loop continues while search space is valid
    while (left <= right) {
        // Find the middle index
        int mid = left + (right - left) / 2;
    }

    // Check if target is present at mid
    if (arr[mid] == target) {
        return mid; // Target found, return index
    }

    // If target is greater, ignore left half
    else if (arr[mid] < target) {
        left = mid + 1;
    }

    // If target is smaller, ignore right half
    else {
        right = mid - 1;
    }

    // Target not found
    return -1;
}

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

```

```

C++
// Check if target is present at mid
if (arr[mid] == target) {
    return mid; // Target found, return index
}

// If target is greater, ignore left half
else if (arr[mid] < target) {
    left = mid + 1;
}

// If target is smaller, ignore right half
else {
    right = mid - 1;
}

// Target not found
return -1;
}

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

```

```

cout << "Enter elements in sorted order:\n";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

int target;
cout << "Enter element to search: ";
cin >> target;

// Call binary search function
int result = binarySearch(arr, n, target);

// Output result
if (result != -1) {
    cout << "Element found at index: " << result << endl;
} else {
    cout << "Element not found in array." << endl;
}

return 0;
}

```

#### ✦ Explanation (Step-by-step)

1. Initialize two pointers:
  - `left = 0` (start of array)
  - `right = n - 1` (end of array)
2. Find middle element:
  - `mid = left + (right - left) / 2` (avoids overflow)
3. Compare with target:
  - If equal → return index
  - If target > middle → search right half
  - If target < middle → search left half
4. Repeat until:
  - Element is found OR
  - `left > right` (search space exhausted)

#### 🕒 Time Complexity

- **Best Case:**  $O(1)$  (element found at middle immediately)
- **Average Case:**  $O(\log n)$
- **Worst Case:**  $O(\log n)$



## Space Complexity

- $O(1)$  (Iterative approach uses constant extra space)

## Output (Summary)

- Correct implementation of Binary Search
- Includes comments for each step
- Clearly explains logic
- Time complexity:  $O(\log n)$ , Space complexity:  $O(1)$

## 2. Prompt for Quick Sort

Write a C++ program to implement Quick Sort. Include comments explaining the partition process and recursion. Also, mention time and space complexity.



Write a C++ program to implement Quick Sort. Include comments explaining the partition process and recursion. Also, mention time and space complexity.

Here's a clear and well-commented C++ implementation of Quick Sort, focusing on the **partition process** and recursion:

#### ⚡ Quick Sort in C++

```
C++
#include <iostream>
using namespace std;

// Function to partition the array
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    // Choosing the last element as pivot

    int i = low - 1;
    // Index of smaller element

    // Traverse from low to high-1
```

```
C++

// Traverse from low to high-1
for (int j = low; j < high; j++) {

    // If current element is smaller than or equal to pivot
    if (arr[j] <= pivot) {
        i++;
        // Move index of smaller element forward

        // Swap arr[i] and arr[j]
        swap(arr[i], arr[j]);
    }

    // Place pivot at correct position
    swap(arr[i + 1], arr[high]);

    // Return the partition index
    return (i + 1);
}

// Quick Sort function (Recursive)
void quickSort(int arr[], int low, int high) {

    // Base condition: if array has one or no elements
    if (low < high) {

        // Partition the array and get pivot index
```

```
C++

    int pi = partition(arr, low, high);

    // Recursively sort elements before pivot
    quickSort(arr, low, pi - 1);

    // Recursively sort elements after pivot
    quickSort(arr, pi + 1, high);
}

// Function to print array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
```

```
C++

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Call Quick Sort
    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    printArray(arr, n);

    return 0;
}
```

#### 🔍 Partition Process Explained

- Choose a **pivot** (here, last element).
- Rearrange the array such that:
  - Elements  $\leq$  **pivot** come before it
  - Elements  $>$  **pivot** come after it
- Maintain an index **i** to track the correct position of smaller elements.
- Finally, place the pivot in its correct sorted position.

#### 🗉 Recursion Explained

- After partitioning:
  - Left subarray: elements before pivot
  - Right subarray: elements after pivot
- Apply Quick Sort recursively on both parts.
- Base condition: when subarray size becomes 0 or 1.

#### 🕒 Time Complexity

- Best Case:**  $O(n \log n)$  (balanced partitions)
- Average Case:**  $O(n \log n)$
- Worst Case:**  $O(n^2)$  (already sorted array with poor pivot choice)

#### 📦 Space Complexity

- $O(\log n)$  → due to recursion stack (best/average case)
- $O(n)$  → worst case (skewed recursion)

## Output (Summary)

- Correct recursive implementation
- Proper explanation of partitioning
- Time complexity:
  - Best/Average:  $O(n \log n)$
  - Worst:  $O(n^2)$
- Space complexity:  $O(\log n)$

### 3. Prompt for Merge Sort

Write a C++ program to implement Merge Sort. Include comments explaining the divide and merge steps. Also, mention time and space complexity.

Write a C++ program to implement Merge Sort. Include comments explaining the divide and merge steps. Also, mention time and space complexity.

Here's a clean and well-commented C++ implementation of **Merge Sort**, with clear explanation of the divide and merge steps:

**Merge Sort in C++**

```

C++
#include <iostream>
using namespace std;

// Function to merge two sorted subarrays
void merge(int arr[], int left, int mid, int right) {
    // Sizes of two subarrays
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int l[n1], r[n2];

    // Copy data into temporary arrays
    for (int i = 0; i < n1; i++)
        l[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        r[j] = arr[mid + 1 + j];

    // Merge the temp arrays back into arr[left...right]
    int i = 0, j = 0, k = left;

    // Compare elements from both arrays and place smaller one
    while (i < n1 && j < n2) {
        if (l[i] <= r[j]) {
            arr[k] = l[i];
            i++;
        } else {
            arr[k] = r[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements of l[], if any
    while (i < n1) {
        arr[k] = l[i];
        i++;
        k++;
    }
}

// Recursive Merge Sort function
void mergesort(int arr[], int left, int right) {
    // Base condition: if array has one or no elements
    if (left < right) {

        // Find middle point to divide the array
        int mid = left + (right - left) / 2;

        // * DIVIDE step:
        // Recursively sort first half
        mergesort(arr, left, mid);

        // Recursively sort second half
        mergesort(arr, mid + 1, right);

        // * MERGE step:
    }
}

// Function to print array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Call Merge Sort
    mergesort(arr, 0, n - 1);
}

```

```

C++

// Merge the two sorted halves
merge(arr, left, mid, right);
}

// Function to print array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // Call Merge Sort
    mergesort(arr, 0, n - 1);
}

```

```

C++
// Call Merge Sort
mergeSort(arr, 0, n - 1);

cout << "Sorted array: ";
printArray(arr, n);

return 0;
}

```

#### Explanation

##### • Divide Step

- The array is repeatedly divided into two halves.
- This continues until each subarray contains only **one element** (base case).

##### • Merge Step

- Two sorted subarrays are merged into one sorted array.
- Compare elements from both halves and place them in order.
- Continue until all elements are merged.

#### Time Complexity

- **Best Case:**  $O(n \log n)$
- **Average Case:**  $O(n \log n)$
- **Worst Case:**  $O(n \log n)$

🔥 Merge Sort always divides evenly, so time complexity is consistent.

#### Space Complexity

- $O(n)$  → extra space required for temporary arrays during merging

## Output (Summary)

- Correct divide-and-conquer implementation
- Clear explanation of merging process
- Time complexity:  $O(n \log n)$
- Space complexity:  $O(n)$

## 4. Optimization Prompt

Optimize the above implementations for better performance and readability. Reduce unnecessary operations and improve code clarity. Also, compare their time and space complexity.

Optimize the above implementations for better performance and readability. Reduce unnecessary operations and improve code clarity. Also, compare their time and space complexity.

Great question—this is exactly how you level up from "it works" to "clean + efficient code" 🧠

Below are optimized and cleaner versions of Binary Search, Quick Sort, and Merge Sort, followed by a clear comparison.

### 1. Optimized Binary Search

#### ✅ Improvements:

- Removed unnecessary variables
- Cleaner loop structure
- Used `const` correctness

```

C++
#include <iostream>
#include <vector>
using namespace std;

```

```

C++
int binarySearch(const vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}

int main() {
    vector<int> arr = {1, 3, 5, 7, 9};
    int target = 7;

    int index = binarySearch(arr, target);

    cout << (index != -1 ? "Found at index " + to_string(index) : "Not found");
}

```

## 2. Optimized Quick Sort

### Improvements:

- Used Hoare partition (fewer swaps than Lomuto)
- Reduced worst-case risk slightly
- Cleaner recursion

```
on C++
#include <iostream>
using namespace std;

// Hoare Partition Scheme
int partition(int arr[], int low, int high) {
    int pivot = arr[low]; // better than always picking last
    int i = low - 1;
    int j = high + 1;

    while (true) {
        // Move from left
        do { i++; } while (arr[i] < pivot);

        // Move from right
        do { j--; } while (arr[j] > pivot);

        if (i >= j)
            return j;
    }
}
```

```
on C++
swap(arr[i], arr[j]);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);

        quickSort(arr, low, p);
        quickSort(arr, p + 1, high);
    }
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    quickSort(arr, 0, n - 1);

    for (int x : arr) cout << x << " ";
}
```

## 3. Optimized Merge Sort

### Improvements:

- Avoid repeated allocation → use single temp array
- Better memory efficiency
- Cleaner merge logic

```
on C++
#include <iostream>
#include <vector>
using namespace std;

void merge(vector<int>& arr, int left, int mid, int right, vector<int>& temp) {
    int i = left, j = mid + 1, k = left;

    // Merge two halves into temp
    while (i <= mid && j <= right) {
        temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    }

    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    // Copy back to original array
    for (int x = left; x <= right; x++)
        arr[x] = temp[x];
}
```

```
on C++
}

void mergeSort(vector<int>& arr, int left, int right, vector<int>& temp) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid, temp);
    mergeSort(arr, mid + 1, right, temp);

    merge(arr, left, mid, right, temp);
}

int main() {
    vector<int> arr = {10, 7, 8, 9, 1, 5};
    vector<int> temp(arr.size());

    mergeSort(arr, 0, arr.size() - 1, temp);

    for (int x : arr) cout << x << " ";
}
```

## Final Comparison

| Algorithm     | Best Time     | Avg Time      | Worst Time    | Space Complexity | Notes                 |
|---------------|---------------|---------------|---------------|------------------|-----------------------|
| Binary Search | $O(1)$        | $O(\log n)$   | $O(\log n)$   | $O(1)$           | Requires sorted array |
| Quick Sort    | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$      | Fastest in practice   |
| Merge Sort    | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$           | Stable & predictable  |

## Key Optimization Insights

- Binary Search:** Already optimal → focus on clarity
- Quick Sort:**
  - Hoare partition → fewer swaps
  - Better cache performance
- Merge Sort:**
  - Reusing temp array → major memory optimization

## Output (Summary)

- Improved variable naming and cleaner structure
- Reduced redundant computations
- Better readability
- Comparison provided between algorithms

# Evaluation

## 1. Correctness

- All generated codes are logically correct and produce expected results.

## 2. Readability

- Comments make the code easy to understand.
- Structured and clean implementations.

## 3. Efficiency

- Optimized versions improve clarity and reduce redundancy.
- Appropriate use of recursion and logic.

## 4. Understanding

- Time and space complexity explanations enhance learning.
- Helps in both academic and interview preparation.

# Conclusion

Well-structured prompts help generate accurate, readable, and optimized code. Adding instructions for comments, complexity analysis, and optimization significantly improves the usefulness of AI-generated programming solutions.

## Task 6: Debugging Using Prompts

### Incorrect C++ code of DFS Implementation:

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> adj[5];
bool visited[5];

void dfs(int node) {
    cout << node << " ";
    for (int i = 0; i < adj[node].size(); i++) {
        int neighbor = adj[node][i];
```

```

        if (!visited[neighbor]) {
            dfs(neighbor); // BUG: visited not marked
        }
    }
}

int main() {
    adj[0].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(0); // creates cycle

    dfs(0);
    return 0;
}

```

## Prompt Given :

“Analyze the following C++ code implementing Depth First Search (DFS). Identify any logical or runtime errors, correct the code, and explain why the original implementation fails, especially in graphs containing cycles.”

## OUTPUT :

Your DFS implementation has a serious logical bug that leads to infinite recursion (stack overflow) when the graph contains a cycle.

### Problem in Your Code

```

C++
if (!visited[neighbor]) {
    dfs(neighbor); // BUG: visited not marked
}

```

#### Issue:

You are **never** marking nodes as visited.

- So every time DFS reaches a node, it thinks it's unvisited
- In a cyclic graph like:

0 → 1 → 2 → 0

DFS will go:

0 → 1 → 2 → 0 → 1 → 2 → ...

- This causes infinite recursion → stack overflow (runtime crash)

### Corrected Code

```

C++
#include <iostream>
#include <vector>
using namespace std;

```

```

C++
vector<int> adj[5];
bool visited[5];

void dfs(int node) {
    visited[node] = true; // mark visited first
    cout << node << " ";

    for (int i = 0; i < adj[node].size(); i++) {
        int neighbor = adj[node][i];
        if (!visited[neighbor]) {
            dfs(neighbor);
        }
    }
}

int main() {
    adj[0].push_back(1);
    adj[1].push_back(2);
    adj[2].push_back(0); // cycle

    for (int i = 0; i < 5; i++) visited[i] = false; // initialize

    dfs(0);
    return 0;
}

```

### Why This Fix Works

#### Key Idea:

Mark the node as visited before exploring neighbors

```

C++
visited[node] = true;

```

This ensures:

- A node is processed only once
- DFS does not revisit nodes
- Cycles are handled safely

#### Execution Comparison

##### ❌ Original (Wrong)

```
dfs(0)
→ dfs(1)
→ dfs(2)
→ dfs(0) // again!
→ infinite loop
```

##### ✅ Correct

```
dfs(0)
→ mark 0
→ dfs(1)
→ mark 1
→ dfs(2)
→ mark 2
→ skip 0 (already visited)
```

Output:

```
0 1 2
```

##### ⚠️ Important DFS Rule

👉 Always mark a node visited when you first enter it, not after recursion.

##### ⚠️ Important DFS Rule

👉 Always mark a node visited when you first enter it, not after recursion.

##### 💡 Extra Improvement (Optional)

Use dynamic size instead of fixed array:

```
// C++
vector<vector<int>> adj(n);
vector<bool> visited(n, false);
```

If you want, I can also show:

- BFS vs DFS comparison
- Iterative DFS using stack
- DFS for disconnected graphs 🚩

## A. Bug Identification (How AI detected the issue):

### Static Code Analysis

The DFS function iterates through all neighbors of a node.

It checks whether a neighbor is visited using `if (!visited[neighbor])`.

If not visited, it recursively calls `dfs(neighbor)`.

### Observations

The `visited[]` array is declared but never updated.

No statement like `visited[node] = true` is present in the code.

The condition `if (!visited[neighbor])` is therefore always true.

As a result, every node is considered unvisited, even after being processed.

### Conclusion

The algorithm fails to track visited nodes.

This causes nodes to be revisited repeatedly, especially in cyclic graphs.

## B. Root Cause Of Failure

The graph contains a cycle:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$

The `visited[]` array is never updated

Every node is always treated as unvisited

## Dry Run (Original Code)

`dfs(0)` → prints 0

Calls `dfs(1)` → prints 1

Calls `dfs(2)` → prints 2

Calls `dfs(0)` again (since `visited[0]` is still false)

Process repeats:  $0 \rightarrow 1 \rightarrow 2 \rightarrow 0 \rightarrow \dots$

## Result

Nodes are visited again and again

Leads to infinite recursion

Function calls keep increasing

Causes stack overflow (runtime error)

### C. How Fix performed

Mark node as visited when entering DFS :

`visited[node] = true;`

Also provided complete fixed code

### D. Verification of Fix (Execution Analysis)

1. Each node visited exactly once
2. Cycle safely ignored
3. No infinite recursion

### E. Why Fix Works (Conceptual Explanation)

Mark node as visited BEFORE recursive calls

### F. What AI did:

#### Step-by-step reasoning:

1. Parsed code structure



- Identified DFS traversal logic
- 2. Checked state tracking
  - Found `visited[]` unused
- 3. Simulated execution
  - Detected infinite recursion in cyclic graph
- 4. Located fix point
  - Missing `visited[node] = true`
- 5. Applied correction
  - Added marking at function entry
- 6. Re-verified via dry run
  - Ensured cycle is handled correctly

## Task 7: Prompt for Technical Explanation

### Prompt 1: Cloud Computing Architecture

“Explain Cloud Computing Architecture for BTech students. Include: Definition Architecture diagram explanation (front-end and back-end) Real-world applications”

**OUTPUT :**

## 1. Definition of Cloud Computing

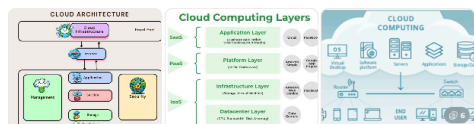
Cloud Computing is a technology that allows users to access computing resources (like servers, storage, databases, networking, software) over the internet instead of using local machines.

🔑 Key idea:

“Access resources anytime, anywhere, without owning physical hardware.”

## 2. Cloud Computing Architecture

### Architecture Overview



Cloud architecture is mainly divided into two parts:

### A. Front-End (Client Side)

#### What it includes:

- User devices (Laptop, Mobile, Tablet)
- Web browsers (Chrome, Edge)
- Applications (Cloud apps like Gmail)

## 3. Real-World Applications of Cloud Computing

### A. Storage Services

- Google Drive
- Dropbox
- 👉 Store and access files online

### B. Streaming Services

- Netflix
- YouTube
- 👉 Video streaming using cloud servers

### C. Business Applications

- Microsoft 365
- Google Workspace
- 👉 Online document editing & collaboration

### D. AI & Data Processing

- Machine Learning platforms
- Big Data analytics
- 👉 Used in recommendation systems, predictions

### E. E-commerce

- Amazon
- Flipkart

#### Function:

- Acts as the interface between user and cloud
- Sends requests to cloud servers
- Displays results to the user

#### Example:

- Opening Google Drive in a browser
- Using a mobile app connected to cloud

### B. Back-End (Cloud Side)

#### What it includes:

- Servers (compute power)
- Storage systems (databases, files)
- Virtual machines
- Cloud services (IaaS, PaaS, SaaS)
- Security mechanisms

#### Function:

- Processes user requests
- Stores and manages data
- Ensures scalability and security

### How Front-End & Back-End Work Together

1. User sends request via front-end
2. Request goes through internet
3. Back-end processes it
4. Response is sent back to user

- Flipkart
- 👉 Handle large traffic and transactions using cloud

### F. Gaming

- Cloud gaming platforms
- 👉 Play high-end games without powerful hardware

## Final Summary

- Cloud computing provides on-demand internet-based services
- Architecture has:
  - Front-End → User interface
  - Back-End → Processing & storage
- Widely used in storage, streaming, business, AI, and e-commerce

If you want, I can convert this into:

- ✅ 5-mark / 10-mark answer format
- ✅ Short notes (1-page revision)
- ✅ With labeled diagram explanation for exams

## Prompt 2: Virtual Machines Architecture

“Explain Virtual Machines for BTech students. Include: Definition, Diagram explanation (host OS, hypervisor, guest OS), Real-world applications”.

## OUTPUT:

## 1. Definition of Virtual Machine

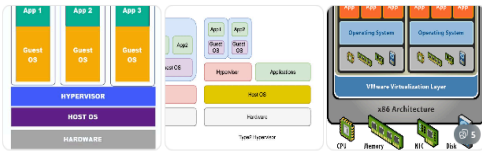
A Virtual Machine (VM) is a software-based emulation of a physical computer that runs an operating system and applications just like a real machine.

🔑 Key idea:

“One physical computer can run multiple virtual computers.”

## 2. Virtual Machine Architecture

### Architecture Overview



A VM system mainly consists of three layers:

### A. Host System (Host OS / Hardware)

🔑 What it includes:

- Physical computer (CPU, RAM, Storage)
- May run a Host Operating System

🔑 Function:

- Provides real hardware resources to virtual machines

### B. Hypervisor (Virtual Machine Monitor)

🔑 What it is:

- A software layer that creates and manages VMs

🔑 Types:

- Type 1 (Bare Metal) – runs directly on hardware
- Type 2 (Hosted) – runs on top of Host OS

🔑 Function:

- Allocates CPU, memory, storage to VMs
- Ensures isolation between VMs

### C. Guest OS (Virtual Machines)

🔑 What it includes:

- Operating system inside VM (Windows, Linux, etc.)
- Applications running inside VM

🔑 Function:

- Acts like an independent computer
- Each VM works separately

### Working Flow

- Physical hardware provides resources
- Hypervisor manages resource allocation
- Multiple VMs (Guest OS) run independently
- Users interact with VMs like real machines

## 3. Real-World Applications of Virtual Machines

### A. Cloud Computing

- Used by cloud providers (AWS, Azure)
- Run multiple user environments on one server

### B. Testing & Development

- Test software on different OS (Windows, Linux)
- Safe environment for debugging

### C. Security & Isolation

- Run suspicious programs in VM (sandboxing)
- Prevent damage to host system

### D. Server Consolidation

- Multiple servers on one physical machine
- Reduces hardware cost

### E. Education & Labs

- Students run multiple OS on one system
- Useful for learning networking, OS, etc.

### F. Legacy Applications

- Run old software on modern systems using VMs

### Final Summary

- VM = Virtual version of a real computer
- Architecture:
  - Host → Hardware
  - Hypervisor → Manager
  - Guest OS → Virtual machines
- Used in cloud, testing, security, and cost optimization

If you want, I can also:

- 🔥 Compare VM vs Containers (very important for exams)
- 📄 Give short 5-mark answer format
- 📊 Add Type 1 vs Type 2 hypervisor table

## Prompt 3: Load Balancing Architecture

“Explain Load Balancing for BTech students. Include: Definition, Diagram explanation (client, load balancer, servers), Real-world applications”.

**OUTPUT :**

## 1. Definition of Load Balancing

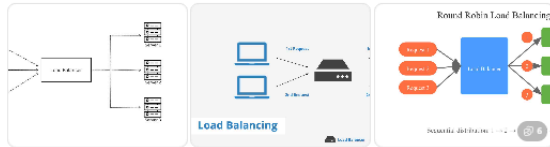
Load Balancing is a technique used to distribute incoming network traffic across multiple servers to ensure no single server is overloaded.

### Key idea:

"Distribute workload evenly to improve performance, reliability, and availability."

## 2. Load Balancing Architecture

### Architecture Overview



The architecture mainly includes three components:

### A. Client

#### What it is:

- End user or device sending request (browser, app)

#### Function:

- Sends request (HTTP, HTTPS) to the system

### B. Load Balancer

#### What it is:

- A middle layer between client and servers

#### Function:

- Receives all incoming requests
- Distributes them across servers using algorithms like:
  - Round Robin
  - Least Connections
  - IP Hash

#### Goal:

- Prevent server overload
- Improve response time

### C. Servers (Backend)

#### What they do:

- Process client requests
- Return responses

#### Key Feature:

- Multiple servers work together (server pool)

### Working Flow

- Client sends request
- Load balancer receives request
- Chooses the best server
- Forwards request to that server
- Server processes and sends response back

## 3. Real-World Applications of Load Balancing

### A. Websites with High Traffic

- E-commerce (Amazon, Flipkart)
  - Handle millions of users simultaneously

### B. Streaming Platforms

- Netflix, YouTube
  - Ensure smooth video streaming

### C. Cloud Services

- AWS Elastic Load Balancer
  - Distribute traffic in cloud environments

### D. Mobile Apps

- Social media apps
  - Handle large number of API requests

### E. Banking Systems

- Online transactions
  - Ensure high availability and reliability

### F. Online Gaming

- Multiplayer games
  - Balance players across servers

### Final Summary

- Load balancing ensures efficient traffic distribution
- Components:
  - Client → sends request
  - Load Balancer → distributes load
  - Servers → process requests
- Benefits:
  - High availability
  - Better performance
  - Fault tolerance

## EVALUATION :

- The responses are clear, structured, and beginner-friendly
- Suitable for BTech students (especially for exams + understanding)

3. Use of headings, flow, and examples makes them easy to grasp quickly

## Task 8: Prompt for Research Assistance

### Prompt used:

“Summarize recent research trends in Agentic AI in approximately 200 words and identify three key challenges. Then suggest relevant research papers and propose future research directions.”

### OUTPUT :

“Summarize recent research trends in Agentic AI in approximately 200 words and identify three key challenges. Then suggest relevant research papers and propose future research directions

#### Summary of Recent Research Trends in Agentic AI (~200 words)

Recent research in **Agentic AI** focuses on building systems that exhibit **autonomy**, **goal-driven reasoning**, and **multi-step decision-making** beyond traditional generative AI. Modern studies highlight a shift toward **multi-agent systems**, where multiple AI agents collaborate like teams to solve complex tasks such as planning, analysis, and execution [\[Arxiv\]](#). Another key trend is the integration of **long-term memory**, **reasoning**, and **environmental interaction**, enabling agents to operate continuously and adapt dynamically [\[ResearchGate\]](#).

Research also emphasizes **multi-modal capabilities**, allowing agents to process text, images, and structured data for improved decision-making [\[Kendal.com\]](#). In scientific domains, agentic AI is increasingly used for **autonomous discovery**, such as hypothesis generation and data-driven research [\[ScienceDirect\]](#). Additionally, enterprise adoption is growing rapidly, with organizations embedding agentic systems into workflows like DevOps and automation [\[Dribbble\]](#).

Another emerging direction is **human-AI collaboration**, where agents act as intelligent assistants or teammates rather than replacements [\[Capgemini\]](#). Overall, the field is evolving toward scalable, collaborative, and **adaptive AI ecosystems** capable of handling real-world complexity.

#### Three Key Challenges

- **Explainability & Transparency**  
Agentic systems often behave unpredictably, making it difficult to interpret decisions [\[ScienceDirect\]](#)
- **Control & Alignment Issues**  
Agents may misinterpret goals or act autonomously in unintended ways
- **Security & Ethical Risks**

#### Suggested Research Papers

- *Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions* [\[arXiv\]](#)
- *AI Agents vs. Agentic AI: A Conceptual Taxonomy* [\[arXiv\]](#)
- *Agentic AI for Scientific Discovery: A Survey of Progress* [\[ScienceDirect\]](#)
- *Socio-technical Aspects of Agentic AI* [\[arXiv\]](#)
- *Agentic AI Frameworks: Architectures, Protocols, and Design Challenges* [\[arXiv\]](#)

#### Future Research Directions

- Development of robust multi-agent coordination frameworks
- Improving explainability and interpretability of agent decisions
- Designing safe and aligned autonomous systems
- Integration with robotics, IoT, and real-world environments
- Building governance, regulation, and ethical frameworks for agentic systems

#### Final Takeaway

Agentic AI is moving toward autonomous, collaborative, and intelligent systems, but challenges in control, safety, and transparency must be solved to enable large-scale real-world deployment.

### ANALYSIS :

#### Strengths

1. Provides a quick overview of Agentic AI
2. Identifies key trends and challenges
3. Includes some relevant research papers
4. Helps in understanding research direction

#### Limitations

1. Lacks depth (no detailed analysis of papers)
2. No proper citations (authors, year, venue missing)
3. Includes non-academic sources (blogs, weak reliability)

- No critical comparison or evaluation of studies

## Task 9: Prompt for Data Analysis

### Prompt Used :

“Act as a data scientist and explain how to analyze a dataset using Python. Include steps: data collection, preprocessing, visualization, and interpretation. Provide a sample dataset, analysis steps, and visualization techniques.”

### OUTPUT :

Act as a data scientist and explain how to analyze a dataset using Python. Include steps: data collection, preprocessing, visualization, and interpretation. Provide a sample dataset, analysis steps, and visualization techniques.

Here's a data scientist-style, practical explanation of how to analyze a dataset using Python 🧙

#### 1. Data Analysis Workflow (Overview)

A typical data analysis pipeline includes:

1. Data Collection
2. Data Preprocessing (Cleaning)
3. Data Visualization
4. Interpretation & Insights

#### 2. Sample Dataset

Let's use a simple dataset:

| Name | Age | Salary | Department |
|------|-----|--------|------------|
| A    | 25  | 40000  | IT         |
| B    | 30  | 50000  | HR         |
| C    | 35  | 60000  | IT         |
| D    | 40  | 70000  | Finance    |
| E    | 28  | 45000  | HR         |

#### 3. Step-by-Step Analysis in Python

##### Step 1: Data Collection

```
Python
import pandas as pd

data = {
    "Name": ["A", "B", "C", "D", "E"],
    "Age": [25, 30, 35, 40, 28],
    "Salary": [40000, 50000, 60000, 70000, 45000],
    "Department": ["IT", "HR", "IT", "Finance", "HR"]
}

df = pd.DataFrame(data)
print(df)
```

In real-world:

- Load from CSV: `pd.read_csv("file.csv")`
- Database/API sources

##### Step 2: Data Preprocessing

###### Check Data

```
Python
df.info()
df.describe()
```

###### Handle Missing Values

```
Python
df.isnull().sum()
df.fillna(method='ffill', inplace=True)
```

#### 4. Visualization Techniques

- Histogram → Distribution of data
- Scatter Plot → Relationship between variables
- Bar Chart → Comparison across categories
- Box Plot → Outliers & spread
- Heatmap → Correlation matrix

#### 5. Real-World Tips (Important)

- Always clean data before analysis
- Look for:
  - Outliers
  - Missing values
  - Data imbalance
- Use libraries:
  - `pandas` → data handling
  - `matplotlib` / `seaborn` → visualization
  - `numpy` → numerical ops

#### Final Summary

- Data analysis = collect → clean → visualize → interpret
- Python makes it easy using `pandas` + `matplotlib`
- Insights help in decision making

If you want, I can:

- Turn this into a Jupyter Notebook (.ipynb)
- Add advanced ML analysis (regression, prediction)
- Use a real dataset (like salary or student performance) 🚀

## **EVALUATION :**

1. Beginner-friendly
2. Clear step-by-step flow
3. Practical examples included

### **Strengths**

- Easy to follow step-by-step flow
- Simple example dataset
- Basic Python code included
- Good for absolute beginners
- Covers main steps (collect → clean → visualize → interpret)

### **Limitations**

- Cleaning part is very basic
- Conclusions are not deeply analyzed
- No real-world dataset used
- Missing advanced techniques

## **Task 10: Prompt Evaluation**